6-3 · Producer / Consumer 애플리케이션 · 정리·회고 0.5H

정리·회고

코드를 리뷰하고, 6-4 CDC 를 위한 준비를 확인

이 시간의 목표

30분 뒤에는

① 빠뜨리면 무슨 일이 나는지 여섯 가지를 코드 리뷰 표로 점검한다

② 오늘의 체크리스트를 스스로 확인한다

③ 자주 겪는 오류 네 가지의 원인과 해결을 안다

④ 6-4 CDC 에 필요한 kafka·MySQL 상태를 확인한다

누적 집계가 재시작하면 0 이 되는 문제 — 상태를 어디에 둘 것인가가 다음 주의 질문

오늘의 요약

코드로 옮기며 확인한 다섯 가지

1. produce() 는 버퍼에 넣을 뿐 — flush() 없이 끝내면 사라진다
2. delivery report 로 건별 성공을, 키 지정으로 같은 파티션을 확인했다
3. 오프셋 커밋 덕에 같은 그룹은 이어읽고, 새 그룹은 처음부터 읽는다
4. 수동 커밋은 유실을 막지만 중복을 남긴다 — 그래서 멱등
5. LAG 은 Consumer 가 꺼지면 쌓이고 켜면 0 으로 줄어든다

코드 리뷰 포인트

빠뜨리면 무슨 일이 — 여섯 가지

빠뜨리면	무슨 일이	고치는 법
flush()	버퍼에 남은 메시지 유실	try/finally 로 감싸기
poll(0)	콜백이 안 불림	루프 안에서 주기적으로
close()	리밸런싱이 늦어짐	finally 에서 호출
키 지정	순서 보장 안 됨	순서가 필요한 단위를 키로
오류 확인	조용히 넘어감	msg.error() 검사
멱등한 수신	재시도 시 중복	UNIQUE 제약이나 MERGE

producer.py · consumer.py 를 열어 이 여섯 줄이 있는지 확인해 보세요

오늘의 체크리스트

여덟 개를 스스로 확인해 보세요

1. produce() 가 버퍼에 넣는 것뿐이라는 것을 이해했다
2. delivery report 로 전송 성공을 메시지 단위로 확인했다
3. flush 를 빼고 실행해 유실을 직접 확인해 봤다
4. acks 세 가지의 차이를 설명할 수 있다
5. 같은 키가 같은 파티션으로 가는 것을 코드 출력으로 확인했다
6. Consumer 재시작 시 이어읽기, 그룹 이름 변경 시 처음부터를 확인했다
7. 수동 커밋으로 바꿔 실패 시 커밋되지 않는 것을 봤다
8. LAG 이 쌓이고 줄어드는 것을 kafka-consumer-groups.sh 로 관찰했다

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
오류 없이 끝났는데 일부(또는 전부) 없음	flush() 없이 종료 — 버퍼가 전송되지 못함	종료 직전 p.flush() · try/finally 로 감싸기
Consumer 가 조용함	reset 기본값 latest · 토픽 이름 오타	earliest + 새 group.id · --list 로 이름 대조
Connection refused	주소 오류(kafka:29092) · compose 내려감	docker compose ps · 호스트는 localhost:9092
수동 커밋이 같은 메시지에서 막힘	항상 실패하는 메시지를 무한 재시도	재시도 횟수 넘으면 DLQ 로 보내고 커밋

호스트 코드에 kafka:29092 를 쓰면 그 이름을 DNS 에서 찾지 못한다 — 6-2 의 리스너 2계층

예상 질문

자주 나오는 질문 네 가지

1. **Producer 를 여러 개 띄워도 되나요?**
   됩니다 — 순서는 파티션 안에서만 보장되므로 같은 키를 여러 Producer 가 보내면 섞일 수 있다

2. **Consumer 가 처리하는 동안 새 메시지는?**
   버퍼에 쌓인다 — 처리가 느리면 LAG 이 커지고, 그때는 파티션과 컨슈머를 늘린다

3. **JSON 말고 다른 형식은?**
   Avro·Protobuf — 스키마를 강제해 「필드 이름 오타」 같은 문제를 막는다

4. **누적 집계를 제대로 하려면?**
   상태를 외부에 저장하거나 Kafka Streams·Spark Structured Streaming 같은 도구를 쓴다

다음 차시(6-4)를 위한 준비

kafka 가 떠 있으면 된다 — MySQL 은 새로 띄운다

1. **2주차 MySQL 컨테이너 확인**
   docker ps -a | grep -E "de-mysql|mysql"

2. **6-2 compose 상태**
   cd ~/data-eng/week6 → docker compose ps — kafka 가 떠 있으면 된다

3. **6-4 에서 추가할 것**
   MySQL 과 Debezium Connect 를 6-4 compose 에 추가 — 2주차 de-mysql 은 그대로 둔다

binlog 는

MySQL 8.0 기본

ON · ROW

다음 회차 예고

6-4 · CDC(Debezium) 실시간 캡처

01

이론 1H

CDC 개념·binlog

Debezium 아키텍처·스냅숏

02

실습 2.5H

커넥터 구축 +

변경 이벤트 분석

03

정리·회고 0.5H

이벤트 구조 정리

체크리스트·질의응답

준비: 6-2 compose 기동 상태 · MySQL 을 같은 compose 네트워크에 컨테이너로 추가 — 별도 설정 없이 CDC 가능

6-3 · 정리·회고 0.5H 마무리

다음 회차는 6-4 CDC(Debezium)

애플리케이션 코드를 한 줄도 고치지 않고, DB 가 이미 남기는 변경 기록(binlog)을 Kafka 로 흘려보냅니다

- ✓ 로그 기반 CDC 가 폴링보다 나은 이유
- ✓ Debezium Connect 기동 · REST 로 커넥터 등록
- ✓ before / after / op 구조 읽기
- ✓ INSERT·UPDATE·DELETE 이벤트 검증

준비할 것

6-2 compose 기동 · MySQL 컨테이너 추가